APPLIED SOFTWARE PROJECT REPORT

OrderFlow: E-commerce Order and Payment Management System

By

MANMOHAN SHARMA

A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial fulfillment of the requirements for the degree of Master of Science in Computer Science

October 2026 (update month/date before submission)

Scaler Mentee Email ID: [ENTER REGISTERED EMAIL]

Thesis Supervisor: Naman Bhalla

Date of Submission: [ENTER ACTUAL SUBMISSION DATE]

WORKING DRAFT — includes evidence placeholders. Replace highlighted fields and insert actual screenshots/benchmark results before submission.

CERTIFICATION

This page follows the provided Applied Software Project Report template. The project guide/supervisor must review and complete this certification according to the institution's process. It is intentionally not signed or represented as approved in this working draft.

I confirm that I have overseen / reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of Master of Science in Computer Science degree. This text must be retained only if the designated supervisor confirms it is accurate and appropriate for final submission.

Naman Bhalla

_____

Project Guide / Supervisor

Date: [SUPERVISOR TO COMPLETE]

DECLARATION

Draft for the candidate to review and personalize. Do not sign unless every statement is true and consistent with Woolf/Scaler academic-integrity rules.

I, Manmohan Sharma, declare that I have reviewed the project implementation and that the report accurately distinguishes implemented and tested functionality from planned work. I will appropriately acknowledge external resources and AI-assisted support used in drafting, code review, diagrams, or documentation, according to the applicable academic policy. I will replace placeholders, verify evidence and references, and ensure all submitted work is an honest account of my contribution before signing.

Project module start date: [ENTER ACTUAL DATE]

Project module end date: [ENTER ACTUAL DATE]

Candidate signature: [SIGN PERSONALLY]

Date: [ENTER DATE]

ACKNOWLEDGMENT

I am grateful to my family for their patience and encouragement throughout this academic project. I also acknowledge the instructors, mentors, and learning community who helped me strengthen my understanding of backend development, software architecture, testing, databases, and deployment practices.

The project benefited from iterative implementation and review: converting a specification into API contracts, separating transport handlers from application services and persistence, introducing transactional inventory reservation, and testing failure and concurrency scenarios. Tools used to assist drafting, code organization, and review should be acknowledged in the final version in accordance with institutional policy.

This page should be personalized to reflect the people and support actually received. No external contribution should be implied if it did not occur.

TABLE OF CONTENTS

- Abstract
- 1. Project Description
- 2. Requirement Gathering
- 3. Use Cases and User Journeys
- 4. High-Level Design (HLD)
- 5. Low-Level Design (LLD) and Class/Component Design
- 6. Database Schema Design
- 7. REST API and DTO Contract
- 8. Feature Development Process
- 9. Transactional Correctness and Concurrency
- 10. Payment Processing and Reconciliation
- 11. Security and Configuration
- 12. Testing Strategy and Results
- 13. Performance Optimization and Benchmark Plan
- 14. Docker, CI and Runtime Verification
- 15. AWS Deployment Flow
- 16. Technologies Used
- 17. Limitations, Risks and Future Work
- 18. Conclusion
- References
- Appendix A: Run and Verification Commands
- Appendix B: Screenshot Placement Checklist
- Appendix C: Requirement-to-Evidence Traceability

LIST OF TABLES

Table 2.01: Functional Requirements - Identity and Catalog

Table 2.02: Functional Requirements - Inventory, Orders and Payments

Table 2.03: Non-functional Requirements

Table 3.01: Actors and Use Cases

Table 6.01: Core Database Tables

Table 6.02: Important Constraints and Indexes

Table 7.01: HTTP Endpoint Inventory

Table 12.01: Current Automated Test Evidence

Table 13.01: Benchmark Methodology and Measurements Needed

Table 16.01: Technology and Rationale

Table 17.01: Gaps and Planned Mitigations

Table C.01: Requirement-to-Evidence Traceability

LIST OF FIGURES

Figure 1.01: Project development and verification lifecycle

Figure 3.01: Use-case overview

Figure 4.01: High-level OrderFlow architecture

Figure 5.01: Low-level order creation flow

Figure 5.02: Order transaction sequence

Figure 5.03: Implemented order/payment state transitions

Figure 6.01: Simplified relational schema and cardinality

Figure 8.01: Order creation request flow

Figure 9.01: Concurrent inventory reservation invariant

Figure 10.01: Payment callback flow

Figure 14.01: Continuous Integration pipeline

Figure 15.01: Proposed AWS target architecture

ABSTRACT

OrderFlow is a backend application for product catalog management, inventory operations, transactional order placement, and simulated payment processing. It is implemented as a modular Go application using Gin for HTTP routing, PostgreSQL for durable relational state, pgx for database access, and JWT-based authentication with role-based authorization. The key engineering concern is coordinating order, stock and payment state without overselling inventory or writing partial records.

The implementation uses a conditional PostgreSQL inventory update inside a transaction, order and line-item snapshots, an inventory movement audit trail, and payment-initiation idempotency. Signed mock-provider callbacks support replay-safe handling of repeated outcomes. Automated tests cover order reservation, insufficient-stock rollback, payment success/failure, idempotency, concurrency and rollback against an isolated PostgreSQL database. The latest previously pushed CI run validates the existing test baseline, migrations, static analysis and Docker smoke check only for the commit it executed.

The residual bundle adds an optional Redis product cache with a TTL and versioned invalidation, a periodic read-only reconciliation reporter, and additional CI/runtime verification. These additions must be treated as implemented-but-pending verification until merged and a new CI run passes. AWS deployment and real payment-gateway integration are not claimed as completed. The report documents the architecture, correctness controls, testing evidence and remaining limitations, and identifies where actual Postman screenshots and benchmark measurements must be inserted before final submission.

1. PROJECT DESCRIPTION

E-commerce backends coordinate several independently changing data sets: product catalogue, stock availability, customer ownership, order lifecycle, and payment outcomes. An unsafe read-then-write inventory sequence can allow two customers to buy the last unit. A failure after stock reservation can leave an orphan reservation. Duplicate payment callbacks can cause repeated transitions unless state and event processing are idempotent.

OrderFlow focuses on these risks in a single-warehouse, single-currency application. The chosen modular-monolith architecture keeps related inventory, order and payment changes inside PostgreSQL transactions. It avoids distributed transaction complexity during the capstone while still allowing the code to separate HTTP transport, application logic and persistence.

Figure 1.01 introduces the component relationships used by the project; the detailed architecture decision and package boundaries are expanded in Chapter 4.

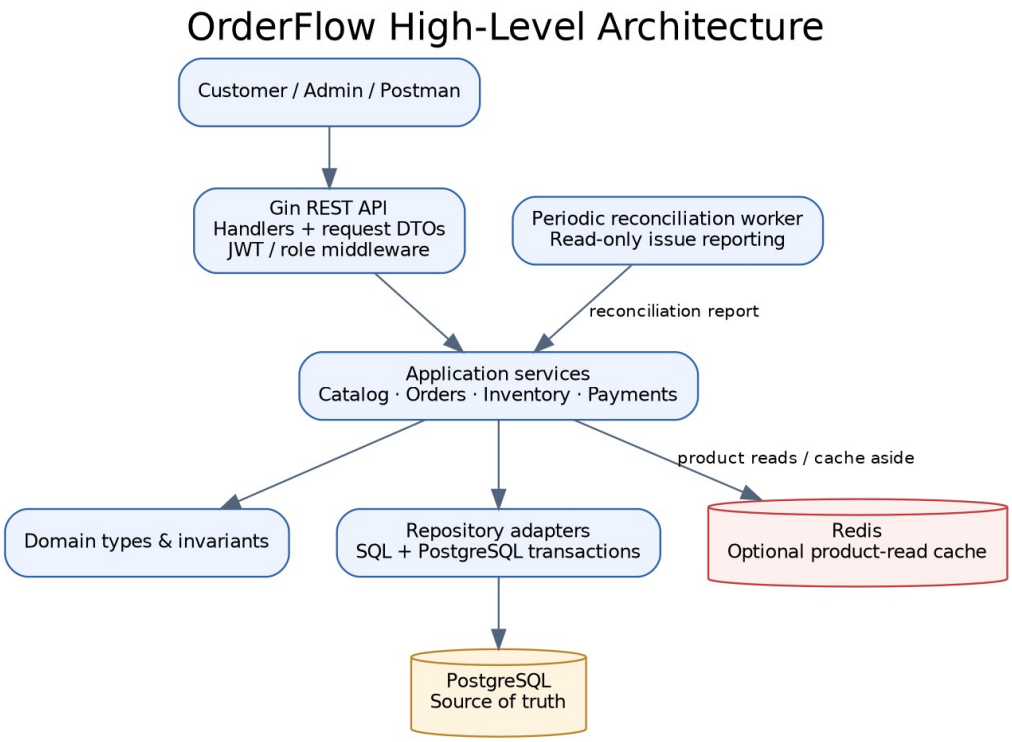


Figure 1.01: OrderFlow component overview

1.1 PROBLEM STATEMENT AND MOTIVATION

A customer request should not supply the price or authoritative total: the backend must load active product data and calculate totals using stored prices. Available stock is on-hand quantity minus already-reserved quantity. A correct order creation operation reserves enough stock and writes the corresponding order, line items, and inventory movement in one database transaction.

Payment initiation can be retried when a client times out or loses a response. The same idempotency key must therefore return the same payment attempt rather than creating duplicates. A provider notification may also be delivered more than once. Repeated delivery must not release inventory twice or turn an already-terminal order into a conflicting state.

These constraints make OrderFlow a practical demonstration of transaction management, relational integrity, repository boundaries, authentication, pagination, idempotency and failure-oriented testing.

1.2 OBJECTIVES, SCOPE AND BOUNDARIES

The primary objective is a testable backend demonstrating secure customer and administrator actions, reliable stock reservation, traceable inventory movements, and safe mock-payment transitions. Secondary objectives include readable API contracts, reproducible local infrastructure, automated integration tests and a report grounded in verified evidence.

In scope are customer registration/login, product catalog and admin CRUD, single-warehouse inventory, authenticated order create/list/detail/cancel, mock payments, signed callbacks, payment reconciliation reporting, migration scripts, unit tests, PostgreSQL integration tests, concurrency tests, OpenAPI and Postman assets.

Out of initial scope are real-money payments, shipping carriers, multiple warehouses, front-end UI, distributed microservices and a production tax engine. Redis and scheduled reconciliation are added as optional acceleration/operations components in the pending residual bundle. Cloud deployment remains a target design until actual AWS resources and evidence exist.

2. REQUIREMENT GATHERING

Requirements were organized into actors, functional behaviors and non-functional constraints. The project specification identifies identity and roles, product/catalog functions, inventory controls, order placement, payment state handling, reconciliation, reporting, performance and deployment. Requirements must be traced to code and evidence rather than marked completed merely because a paragraph exists in a report.

The main acceptance criterion for correctness is that multiple concurrent requests cannot reserve the same available inventory unit. Another is that repeated callbacks with the same terminal outcome cause no second inventory release or duplicate transition. The dedicated test database is named orderflow_test, and the integration helper refuses to run against any other database name.

Category	**Requirement / acceptance condition**
Identity	Unique email; bcrypt-hashed password; JWT; CUSTOMER and ADMIN roles.
Catalog	Active product reads; unique SKU; server-owned price; admin create/update/deactivate.
Inventory	Stock receipt/adjustment with reason; availability; audit movement records.
Orders	Atomic stock reservation; stored price snapshots; owner-scoped history/detail/cancel.
Payments	Idempotent initiation; signed mock callback; replay-safe terminal transitions; reconciliation.
Quality	Unit + PostgreSQL integration + concurrency tests; Docker/CI; honest performance evidence.

2.1 FUNCTIONAL REQUIREMENTS - IDENTITY AND CATALOG

The registration API creates customer accounts with a secure password hash. User-provided role values are not accepted during registration; new registrations begin as CUSTOMER. Login verifies the supplied password and returns a signed access token with a finite expiry. Protected endpoints require a bearer token, and admin-only operations check the role claim before the handler executes.

The product catalog provides public reads of active products, including search and pagination. Product create/update/deactivation is protected by ADMIN authorization. Prices are stored in integer paise to avoid floating-point money arithmetic. A unique database constraint on SKU prevents duplicate product codes even when concurrent requests race.

The product detail handler validates UUID-shaped identifiers. The service performs business validation, and the repository owns SQL and persistence mapping. These layers reduce the risk of mixing JSON binding, HTTP status handling, business rules and SQL in one function.

ID	Requirement	Priority
FR-01–05	Registration, login, roles, authorization and password hashing	Must
FR-06–08	Admin product CRUD, unique SKU and backend-owned price	Must
FR-10	Public active-product/catalog reads	Must

2.2 FUNCTIONAL REQUIREMENTS - INVENTORY, ORDERS AND PAYMENTS

Inventory APIs expose on-hand, reserved and available quantities. Administrators can receive stock and make controlled adjustments with an explanatory reason. Inventory movements preserve the quantity delta, resulting quantity, actor and a reference to the related order when applicable.

Order APIs create an order and order-item snapshot, list the current customer's orders, return details only for the owner, and allow cancellation while the order is eligible. The currently implemented order request shape represents one product per request; the original specification's multi-product requirement remains a gap unless it has been added and tested in a later commit.

Payments are simulated, not real-money transactions. Initiation accepts an idempotency key. A webhook-like callback validates HMAC-SHA256 over the exact body bytes and uses an event identifier for replay safety. The current callback contract must still be reviewed against the specification's amount/currency-match requirement. Reports and state transitions should be tested through both service integration tests and HTTP-level requests.

ID	Requirement	Status at report draft
FR-09,12	Stock receipt and audit history	Implemented and previously runtime-tested
FR-13,20	Multi-item order and order idempotency	Verify/fix before final acceptance
FR-21–27	Mock payments, callbacks, idempotency, amount/currency checks, failure reason and reconciliation	Partially implemented; amount/currency/failure details need review
FR-28–30	Confirm/consume stock; cancel/expiry releases reservation	Confirm success stock consumption and expiry behavior
FR-31–32	Admin order/payment/inventory/sales reports	Partial; reconciliation and inventory movement views exist

2.3 NON-FUNCTIONAL REQUIREMENTS

Data consistency: order and inventory writes must be atomic. Concurrency: reservation uses a conditional UPDATE against the current available quantity, avoiding a read-then-write check. Security: tokens and webhook secrets are not committed, passwords are hashed, and authorization is enforced server-side.

Maintainability: handler DTOs, domain types, services and repositories have distinct responsibilities. Reliability: PostgreSQL is the source of truth; Redis, when enabled, is a best-effort cache and must not gate product API availability. Operability: health/readiness checks and the reconciliation reporter support diagnostics.

Performance should be measured rather than asserted. The specification provides a target p95 under 300 ms for core DB-backed APIs under a defined local load, excluding external provider delays. No measured p95 or throughput value should appear in the final report until a reproducible benchmark is run and its environment, data volume, concurrency, duration and result are captured.

Quality attribute	**Control / evidence**
Consistency	PostgreSQL transaction; database constraints; rollback integration test.
Concurrency	Parallel order/payment integration tests; race detector.
Security	JWT + roles; bcrypt; HMAC callback; local secret hygiene.
Performance	Indexes, pagination, optional product caching; benchmark still required.
Observability	Health/readiness; logs; scheduled reconciliation issue reporting in pending bundle.
Maintainability	Go package boundaries, repository interfaces, OpenAPI, CI workflow.

3. USERS AND USE CASES

The Customer actor registers or logs in, browses active products and stock availability, creates an order, lists their own orders, opens a specific owned order, cancels eligible pending orders, and initiates a simulated payment. The customer is not allowed to create products, adjust inventory or view administrator reconciliation data.

The Administrator manages the catalog, records stock receipts and adjustments, inspects inventory movement history and reads reconciliation reports. Admin capability is represented by a role claim and enforced by middleware before handlers are called.

The simulated payment provider is represented by a callback endpoint whose request body is signed with a shared secret. It is not a production gateway and does not handle real card data or money movement.

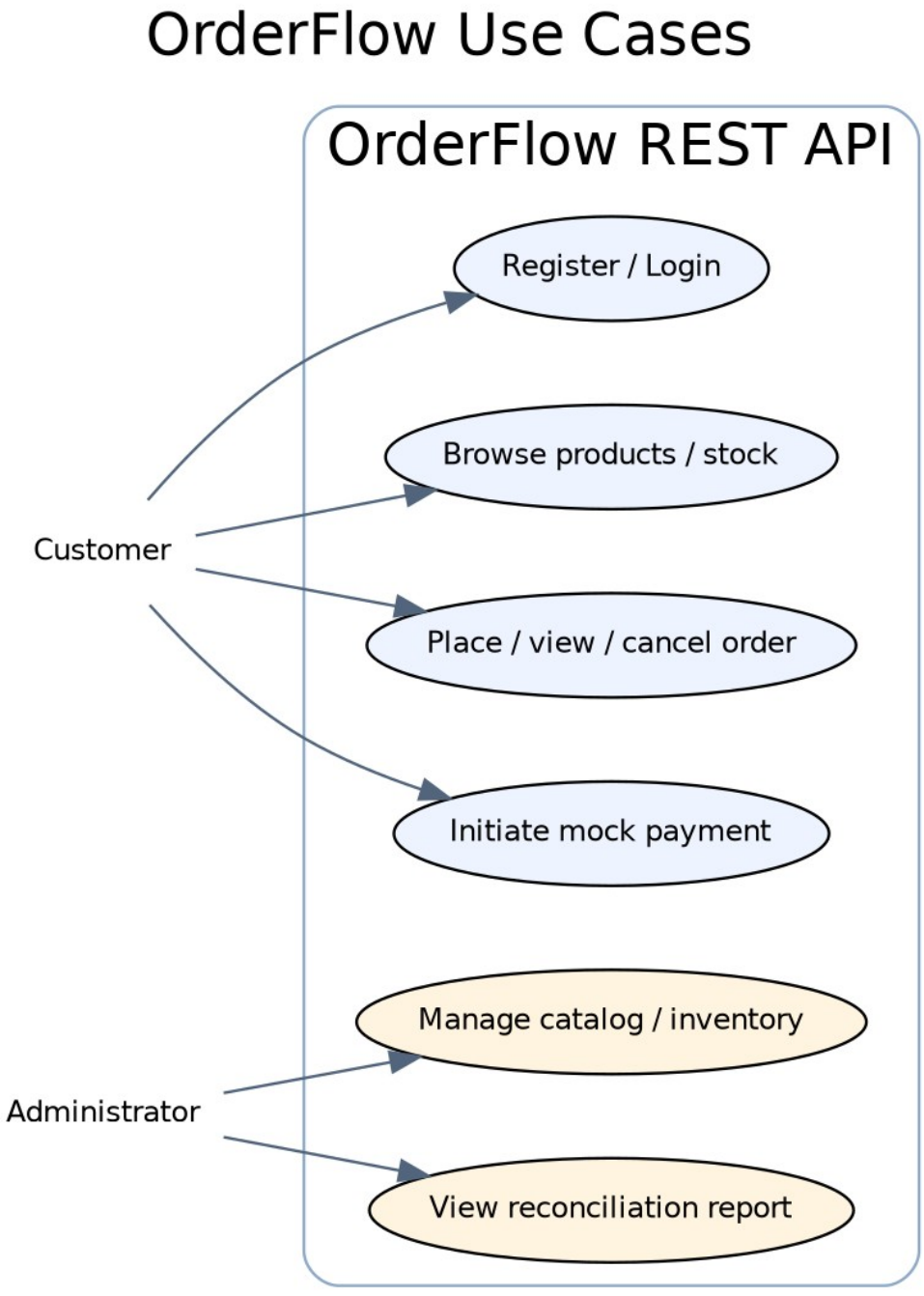


Figure 3.01: OrderFlow actors and use cases

4. HIGH-LEVEL DESIGN (HLD)

The application is a modular monolith. One Go API process exposes HTTP routes through Gin, validates requests, runs authentication/role middleware, invokes application services and persists state using PostgreSQL repositories. This is a deliberate choice: order and inventory writes need one transactional boundary, and distributing them among independent services would introduce network failure and distributed consistency complexity that is not required for this capstone.

PostgreSQL is authoritative for products, orders, order items, payments, inventory balances and movement history. Redis is an optional read-through/cache-aside layer for catalog list and item reads in the residual bundle. A cache outage should fall back to SQL. The reconciliation worker calls existing reporting logic periodically and logs detected issues; it does not settle transactions automatically.

The diagram describes the intended source architecture. It must not be presented as proof of AWS deployment. The AWS figure later in this report is explicitly a proposed target.

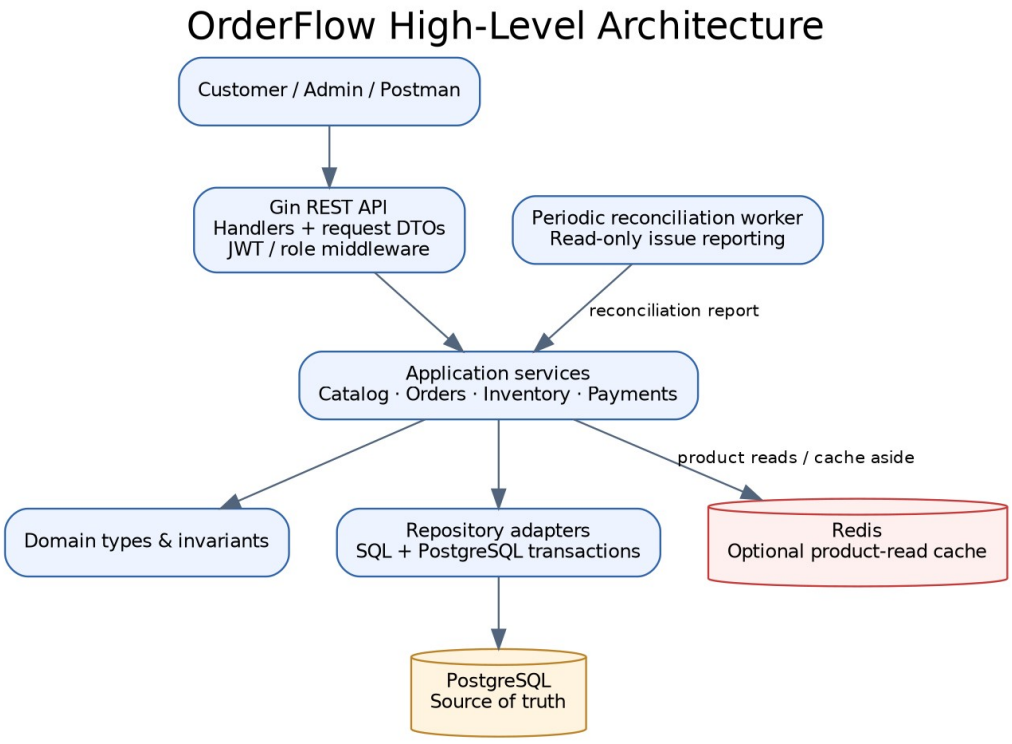


Figure 4.01: High-level OrderFlow architecture

4.1 ARCHITECTURE DECISIONS AND TRADE-OFFS

A modular monolith keeps the order state machine, payment callback and inventory update within a single executable and shared relational transaction. It is easier to test on a laptop than an event-driven collection of microservices and is appropriate for the scope and time limits of an applied capstone.

The repository pattern isolates SQL from services. This improves testability and keeps HTTP transport concerns out of business logic. The current interfaces are concentrated where useful, including the product repository adapter for optional caching. We do not create an empty generic abstraction for every type just for pattern compliance.

PostgreSQL row locking and conditional updates protect state transitions. Redis is not used for stock counts, payment status or authorization because those values require durable consistency. Cache entries are disposable and can always be reconstructed from SQL.

5. LOW-LEVEL DESIGN (LLD)

The handler parses path, query and JSON input, verifies a token through middleware, and maps domain errors to suitable HTTP status codes. The handler should not implement multi-step business transactions or contain SQL. Services perform validation, calculate totals from stored product values, coordinate repositories and decide which state transitions are permitted. Repositories encapsulate transaction operations, SQL predicates and row-lock semantics.

Domain types represent products, orders/items, payments, inventory summaries/movements and reconciliation issues. HTTP request DTOs are private structs near their handlers; that is a valid lightweight boundary for this project. A separate DTO package may become useful for a larger public API, but duplication is not a project requirement.

The current order service uses one transaction to reserve inventory, insert the order, create the item snapshot and record an audit movement. On error, deferred rollback prevents partial database writes. The test suite explicitly verifies insufficient stock leaves no partial order.

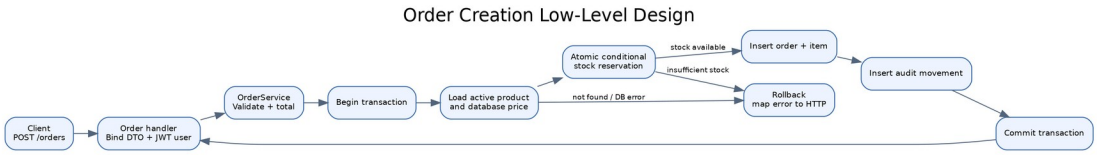


Figure 5.01: Low-level order creation flow

5.1 ORDER CREATION REQUEST FLOW

The client supplies a product identifier and quantity in the currently implemented API. The client does not supply the authoritative price or total. The handler binds and validates the request, JWT middleware identifies the customer, and the service rejects invalid user/product IDs and out-of-range quantities.

The service begins a transaction, loads an active product and its stored price, calculates the total in integer paise, and calls an atomic inventory reservation statement. If stock is unavailable, the service returns an insufficient-stock error; the transaction rolls back. If reservation succeeds, the order and order item are written using a snapshot of the product name and unit price, and an ORDER_RESERVED movement is recorded before commit.

Before final submission, the original requirement for multiple products in one order and an order-level idempotency key must be reconciled with the API implementation. The acceptance test should ensure that an error on any line rolls back reservations for every line.

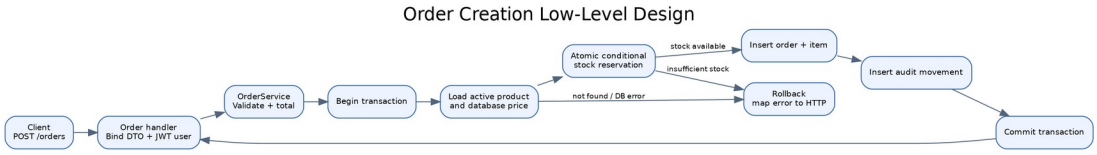


Figure 5.02: Order transaction request flow

5.2 PAYMENT STATE TRANSITIONS

Payment initiation applies a per-request idempotency key and prevents duplicate concurrent pending attempts for the same order. The mock callback reads the exact raw request body, verifies its HMAC-SHA256 signature in constant time, then passes the requested outcome and event ID to the payment service.

The repository locks the order before the payment, checks the current state, and makes terminal callbacks replay-safe. A failure callback releases the associated reserved stock and records an ORDER_RELEASED inventory movement in the same transaction. A success callback confirms the order. The current test evidence for success historically expected the stock reservation to remain; the project specification expects the reserved stock to become consumed. That behavior must be explicitly aligned and tested before report finalization.

The system only has a simulated provider; there is no Stripe SDK, real acquiring bank or real-money settlement. This boundary is documented to avoid overstating the implementation.

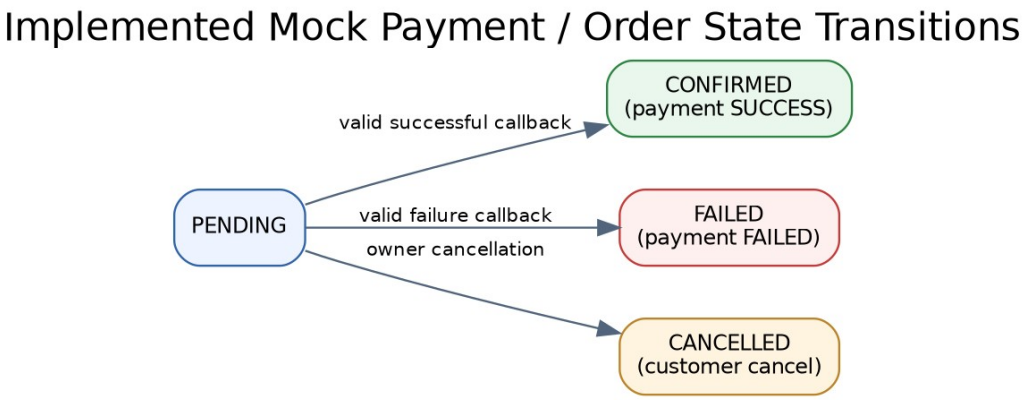


Figure 5.03: Current mock payment and order state transitions

6. DATABASE SCHEMA DESIGN

The data model separates catalog/product definition from current inventory, order headers from line-item snapshots, and current order/payment status from an append-oriented inventory movement history. UUID keys identify records. Foreign keys prevent orphan items, payments and inventory rows. Unique constraints protect email and SKU values. Check constraints reject negative prices, invalid quantities and reserved inventory exceeding on-hand inventory.

An order item retains the product name, quantity, unit price and line total at order time. This snapshot is important because a product can be renamed or repriced after an order is placed. Payments reference orders, while inventory movements reference products and optionally an order identifier.

The simplified relational model is presented in Figure 6.01. The production schema is defined by ordered SQL migrations under migrations/. The separate integration-test database uses the same migration chain and should never be replaced with the development or production connection string.

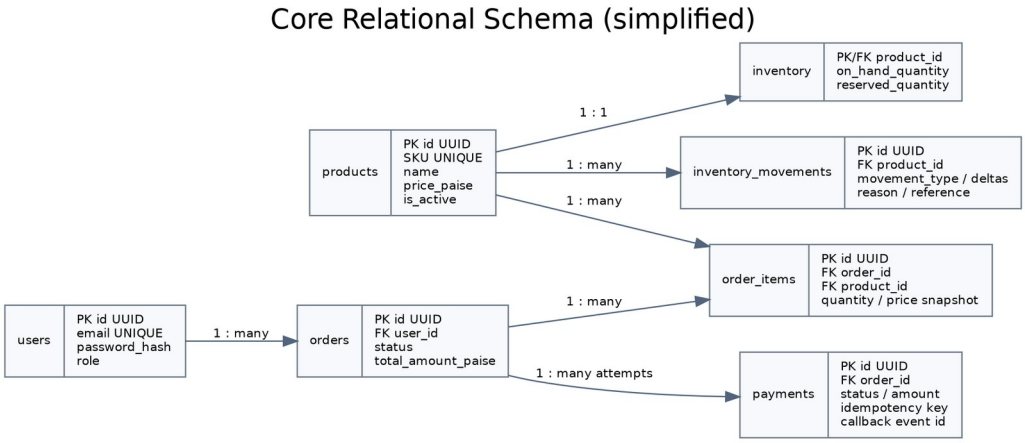


Figure 6.01: Simplified relational schema and cardinalities

6.1 USERS, PRODUCTS AND INVENTORY TABLES

users stores the normalized email identity, bcrypt hash and role. Passwords must never be returned in API payloads. products stores SKU, name, optional description, price in paise, active status, and timestamps. Deactivation is used instead of hard deletion so historical order item snapshots and movement records remain explainable.

inventory has one row per product in the current single-warehouse model. on_hand_quantity is physical stock on hand, reserved_quantity is stock committed to pending or confirmed workflow states depending on the chosen fulfillment model, and available quantity is calculated as on-hand minus reserved. The database has constraints ensuring all counts are non-negative and reserved does not exceed on-hand.

The inventory model must be treated as a single warehouse. Supporting multiple warehouses would require a warehouse dimension and a composite key for inventory balances, not simply another column in the current record.

Table	**Purpose**	**Important integrity rules**
users	Customer/admin identity	Unique email; role in CUSTOMER/ADMIN
products	Catalog and server-owned price	Unique SKU; non-negative price
inventory	Current stock state	Non-negative quantities; reserved <= on-hand

6.2 ORDERS AND ORDER ITEMS

orders stores ownership, customer email snapshot, status, total amount in paise and timestamps. It is the aggregate root for order lifecycle. Current statuses include PENDING, CONFIRMED, CANCELLED and FAILED. State updates must be guarded so callbacks and cancellation cannot independently overwrite each other after the order has left PENDING.

order_items references the parent order and product, but also stores the product name and amount snapshot. Each row has positive quantity and non-negative unit and line totals. Multiple items per order are required by the original project specification; if implementation still accepts only one item per request, the report must label multi-item ordering as an open requirement rather than claim coverage.

Order list/detail queries filter by user ownership. The list query retrieves an item join in one query after pagination and a separate count query, avoiding a query per order item (the N+1 anti-pattern). Sorting includes stable tie breakers so pagination is deterministic under equal timestamps.

Table	**Key fields / semantics**
orders	id, user_id, status, total_amount_paise, timestamps
order_items	order_id, product_id, product_name snapshot, quantity, unit_price_paise, line_total_paise
Important rule	The API loads prices from products and never trusts client-submitted totals.

6.3 PAYMENTS AND INVENTORY MOVEMENTS

payments stores order association, provider, provider payment reference, status, amount and timestamps. The workflow migration adds idempotency and callback event identifiers with unique indexes. These indexes make uniqueness a durable database property rather than a process-local map.

inventory_movements records movement type, deltas, resulting balances, reason, actor, reference type/reference ID and timestamp. Receipts, adjustments, reservations and releases should be explainable by reviewing the movement sequence. The movement table supports audit and investigation, but it is not a substitute for enforcing inventory invariants in SQL.

Failure reason and payment amount/currency validation were explicit concerns in the original specification. The current schema/callback model should be reviewed for these fields before claiming full requirement coverage.

Table	Purpose	Example constraints/indexes
payments	Payment lifecycle and callback correlation	Unique provider reference, idempotency key, callback event ID
inventory_movements	Auditable stock changes	Product/time and movement-type/time indexes
orders/order_items	Business relationship	FKs; status and quantity checks

6.4 MIGRATIONS, ROLLBACK AND INDEX STRATEGY

Migrations are ordered with numeric prefixes and `.up.sql` / `.down.sql` names. The CI workflow creates a disposable PostgreSQL database, applies every up migration in lexical order, and runs tests against that database. A developer must not rerun the initial migration chain against an already-initialized database; schema rollback/repair must be done only after confirming the database name and reviewing its contents.

Indexes support common access patterns such as user order history, order-item joins, payment by order/status/time, and movement history by product/time. Index selection should follow measured query patterns. Excess indexes add write overhead and storage, so new indexes should be justified through EXPLAIN (ANALYZE, BUFFERS) against representative data.

Partial indexes can enforce uniqueness only for non-null idempotency identifiers and reduce index entries when the schema permits null values for old records. Any new migration added after the currently initialized local test DB must be applied as that migration only, not by replaying all prior migrations.

7. REST API DESIGN AND DTO CONTRACT

The API is JSON over HTTP. Public routes are health/readiness, authentication, active product reads and inventory availability. Bearer JWT protects profile/order operations, and ADMIN role gates product mutation, stock changes and reconciliation. The complete OpenAPI description is maintained under docs/openapi.yaml and Postman requests are in docs/postman/OrderFlow.postman_collection.json.

Handlers bind JSON into request DTOs and validate required fields, format, UUID identifiers and quantity limits. Services repeat business-critical checks because transport validation is not a security boundary. Responses use integer paise for monetary values and stable identifiers/timestamps. Error mapping avoids exposing database messages or stack traces.

The existing API does not currently expose a built-in Swagger UI endpoint. The OpenAPI file can be opened in Swagger Editor or Swagger UI separately. API documentation must match the route registration and actual JSON formats; before final submission, test each documented request with the local app and update any discrepancy found.

7.1 API ENDPOINT INVENTORY

The endpoints below describe the current API surface on the remote baseline; always re-check after merging the residual bundle. Public access is intentionally limited. An authenticated customer can view only their own order collection and order IDs. Administrator endpoints require both a valid token and the ADMIN role.

Method and route	Access	Purpose
GET /health	Public	Process health
GET /ready	Public	PostgreSQL readiness
POST /auth/register	Public	Register CUSTOMER
POST /auth/login	Public	Get JWT
GET /products	Public	Search and paginate active products
GET /products/{id}	Public	Active product detail
POST/PATCH/DELETE /products[/{id}]	ADMIN	Create/update/deactivate product
GET /inventory/{product_id}	Public	Inventory availability
POST /orders	Bearer JWT	Create order/reserve stock
GET /orders, GET /orders/{id}	Bearer JWT	Owned order history/detail
POST /orders/{id}/cancel	Bearer JWT	Cancel eligible pending order
POST /orders/{id}/payments	Bearer JWT + Idempotency-Key	Initiate mock payment
POST /payments/mock-callback	HMAC signature	Mock provider callback
GET /admin/payments/reconciliation	ADMIN	Report stale/inconsistent states

7.2 API ERROR HANDLING AND OWNERSHIP

Invalid transport shapes are rejected with 400. A missing/invalid token returns 401, an insufficient role returns 403, a missing/inaccessible resource returns 404, and state/inventory/idempotency conflicts return 409. Unexpected failures return a generic 500 error to prevent implementation details from leaking. A production API should consider a consistent machine-readable error envelope with stable error codes and request IDs.

Ownership is enforced by querying order ID and user ID together. Hiding an order owned by another user as not found avoids leaking resource existence. The service also checks customer identity at payment initiation by locking the target order and comparing its owner. Authorization should be tested for both successful own-resource access and denied cross-customer access.

Pagination uses bounded page size and stable order by columns. API evidence screenshots should redact bearer tokens and secrets; Postman environments should use variables rather than hard-coded real credentials.

8. FEATURE DEVELOPMENT PROCESS: TRANSACTIONAL ORDER CREATION

The critical feature is order placement. A client requests a quantity for an active product. The application loads the server price, computes the total in paise, and attempts a conditional update that only increments reserved quantity when the remaining quantity is enough. Because the condition is evaluated atomically by PostgreSQL, concurrent requests do not both read the same availability and reserve it twice.

The transaction writes all dependent records or none: inventory reservation, order header, order item price/name snapshot and an ORDER_RESERVED inventory movement. If a later statement fails, transaction rollback restores the reservation. This is stronger than issuing independent updates and attempting compensating writes after failure.

The test suite includes a limited-stock concurrency scenario and a rollback scenario where the order cannot be created for an absent user after the reservation attempt. The expected assertions include the number of successful orders, reserved quantity, order count and movement count.

8.1 REQUEST PAYLOAD AND SERVICE FLOW

The currently implemented request is a JSON object with product_id and quantity. The service boundary accepts a domain request rather than the Gin context. This separation prevents an HTTP library dependency from leaking into business rules and makes the service directly testable in integration tests.

The repository encapsulates BeginTx, active-product loading, conditional inventory reservation, order insertion, order-item snapshot insertion and movement insertion. Each repository method accepts the request context and transaction when part of a multi-write unit. The service commits only when every required write succeeds.

The initial single-product API works for common checkout examples but does not fulfill the full original one-or-more-products requirement. The final requirement traceability must indicate whether multi-line order support was added in a later commit or is a consciously deferred limitation.

9. TRANSACTIONAL CORRECTNESS AND CONCURRENCY

Concurrency tests use independent goroutines and a shared dedicated database. With five available units and ten competing quantity-one requests, exactly five orders should succeed and reserved stock should end at five. The database must never allow reserved_quantity to exceed on_hand_quantity. These tests demonstrate an invariant under concurrent service calls rather than only proving behavior in a serial test.

Payment initiation locks the order before creating or returning a payment attempt. This serializes duplicate initiations against the same order. Callback processing uses a consistent order-then-payment lock order to reduce deadlock risk. Same-outcome callback replay returns the terminal payment rather than repeating the stock transition.

Rollback is a first-class behavior. A service must not leave a partially-created order, an inventory movement without its matching stock update, or a reserved-stock counter changed after the transaction fails. Tests should inspect all affected tables, not only the API status code.

9.1 TEST CASES AND ACCEPTANCE CRITERIA

The tests already committed cover order reservation, rejection under insufficient stock, payment success/failure, idempotency, concurrent reservation, concurrent payment initiation, concurrent callback replay and a rollback path. The residual code adds Redis adapter/TTL and cache invalidation tests plus a worker test. Those Redis-specific scenarios execute only when a test Redis endpoint is configured; updated CI should configure it explicitly.

The strongest concurrency acceptance test is: with five units available and ten competing one-unit requests, there must be exactly five successful order results, exactly five order records for the fixture owner and exactly five units reserved. A duplicate failed callback should produce exactly one ORDER_RELEASED movement and leave reserved stock at zero for that order.

Run `go test -count=1 ./...`, `go test -race -count=1 ./internal/integration/...`, `go vet ./...`, and `git diff --check`. These results should be taken from the latest branch state. A green CI run is valid evidence only for the specific commit SHA shown in that run.

Test category	**Expected evidence**
Order transaction	Order, item and movement commit together.
Insufficient stock / rollback	No partial order; inventory unchanged.
Concurrency	No oversell; exact successful reservation count.
Payment idempotency	Repeated key returns same payment record.
Callback replay	No duplicate stock release or transition.
Redis cache	TTL expiry and mutation invalidation; optional service fallback.
Worker	Periodically reports reconciliation issues and exits on context cancellation.

10. PAYMENT PROCESSING AND RECONCILIATION

The capstone uses a simulated payment provider by design. Payment initiation writes a PENDING payment record, protects the operation with an idempotency key, and returns the existing attempt on retry. The callback endpoint verifies a signature over the exact request bytes before parsing data. This stops unauthenticated clients from directly marking their own order paid.

The payment repository locks the order and payment rows, rejects conflicting transitions, and applies success/failure updates in the same transaction. On failure, all order item reservations are released and movement history is updated once. On success, the report should align the implementation with the specification's inventory-consumption requirement; the previously verified code path retained the reservation, which is a specific gap to resolve or explicitly model as a separate fulfillment phase.

The reconciliation report currently detects stale pending payments, confirmed orders lacking successful payments and failed payments whose orders remain pending. The new worker periodically emits issues to logs but does not automatically query a live gateway or mutate payment state. Therefore it is a read-only monitor, not a full payment recovery engine.

10.1 PAYMENT EDGE CASES AND LIMITATIONS

Important payment edge cases include duplicated event IDs, a callback arriving after cancellation, mismatch between callback amount and order amount, unknown or missing provider state, an unavailable database, and a caller retrying initiation after a timeout. A safe implementation should not infer success or failure from an unknown outcome. Production recovery needs provider-native lookup, retry policy, signing keys, event storage and observability.

The current mock callback DTO has payment ID, outcome and event ID. The original specification asks for amount/currency validation and a failure reason. Until those validations and fields are implemented and tested, they must be noted as incomplete rather than described as satisfied. No real payment credentials or card details belong in the repository.

For a production provider, the application must verify its exact documented signature format, deduplicate provider event IDs, verify payment reference, amount and currency, and route callback and reconciliation through one shared state-transition path.

11. SECURITY AND CONFIGURATION

Passwords are hashed with bcrypt. Access tokens are signed using an environment-provided JWT secret and carry user ID/role claims with expiration. Callback authentication uses a separate mock-payment shared secret. The local `.env` file is untracked and must never be copied into reports, screenshots, final ZIPs or remote commits.

Defense in depth includes transport-level DTO validation, service-level input validation, SQL parameters rather than string interpolation of untrusted values, database constraints, owner-scoped queries and role middleware. In the sample local configuration, the secrets must be independently generated, at least the configured minimum length and changed from placeholders.

Before a public AWS deployment, enforce HTTPS, store secrets in managed secret storage, limit ingress/egress using security groups, use private database/cache subnets, rotate secrets, enable database backups, and avoid logging authorization headers or raw webhook secrets. The application should not be publicly exposed until production-specific controls have been reviewed.

12. TESTING STRATEGY AND CURRENT RESULTS

Testing consists of unit tests for request validation and middleware, service validation tests, PostgreSQL-backed integration tests for transaction outcomes, concurrency/rollback tests, and runtime checks for key endpoints. The integration tests deliberately require a database whose name is exactly orderflow_test, limiting risk to development data.

At the last confirmed remote commit 60c255b, GitHub Actions passed migration application, unit/database tests, race-enabled integration tests, vet, whitespace check, container build and `/health`/`/ready` probes. That evidence predates the residual Redis/worker overlay. The updated workflow must run and pass after the new bundle is committed before those additions can be described as CI-verified.

No coverage percentage should be stated without rerunning `go test -cover ./...` at the final commit. Similarly, no performance improvement in milliseconds or percentage should be claimed until a benchmark before/after cache or query optimization is measured with a defined environment and repeatable workload.

12.1 VERIFICATION COMMANDS AND EVIDENCE

The repeatable local checks are `go test ./...`, `go test -race ./internal/integration/...`, `go vet ./...` and `git diff --check`. The residual bundle includes a script that runs `go mod tidy`, gofmt, the tests, race tests and vet while requiring the dedicated integration DB and launching Redis with Compose.

Capture a screenshot of the full terminal command and successful summary only after running the commands on the actual development machine. Avoid showing DSNs, passwords, JWTs, webhook signatures or `.env` content. In the report, place the screenshot under the testing/evidence subsection with the caption “Figure 12.01: Local verification commands passed on [date]”.

Keep the remote CI URL and commit SHA in the evidence folder. A CI check run from a prior SHA is valuable evidence of the prior code state but does not verify files that were added later.

13. PERFORMANCE OPTIMIZATION AND BENCHMARK PLAN

Performance optimization must start with a defined baseline. Use a fixed product/order dataset and stable local environment, warm the application, run a concurrency level representative of the target, and repeat several samples. Record median and p95 latency, throughput, error count, Go version, CPU/memory, Docker resource limits, database version and Redis status. Keep raw output and commands with the report.

The product list query currently filters active rows, optionally searches name/SKU and paginates. Order history fetches a page of order IDs and line items via a join plus a count query rather than issuing an N+1 query for every order. Indexes support user/time order history and movement/payment lookups. SQL query plans should be inspected with EXPLAIN ANALYZE before adding indexes.

The specification target is p95 below 300 ms for core database-backed APIs under defined local load, excluding provider latency. Until benchmark output exists, report this as an acceptance target, not as an achieved result.

Measurement	**Required fields**
Baseline	Build/commit, dataset size, concurrency, duration, environment, DB state.
Cache on/off	Same requests and dataset; warm-up; p50/p95; throughput; errors.
Query plan	EXPLAIN ANALYZE / buffers for catalog and order-history queries.
Acceptance	p95 target <300 ms only if the measurement supports it.

13.1 REDIS CACHE DESIGN (PENDING VERIFICATION)

The residual bundle adds optional caching for product list/search and product detail reads. PostgreSQL remains the source of truth. Cache keys include a generation number; successful product create/update/deactivation increments the generation so newly issued reads do not reuse keys from a previous generation. Each value has a configurable TTL, defaulting to 60 seconds.

The Redis adapter uses a dedicated client with finite dial/read/write timeouts and disabled automatic retries. The application pings Redis during startup; if unavailable, cache integration is disabled for that process and product reads continue through PostgreSQL. This fail-open design avoids turning a cache outage into a product-catalog outage.

The residual test bundle uses Redis DB 14 for cache adapter TTL/version checks and DB 15 for PostgreSQL-plus-Redis invalidation integration tests. CI should provide a disposable Redis service. The cache is verified only after `go mod tidy`, all unit/integration tests, race checks, Docker smoke test and latest CI all pass on the commit.

14. DOCKER AND CONTINUOUS INTEGRATION

Docker packages the Go application into a small multi-stage container. The build stage compiles a static Linux binary. The runtime stage contains the binary and CA certificates, runs as a non-root user, and exposes port 8080. The `.dockerignore` excludes local credentials, Git state and unrelated files from the build context.

CI means Continuous Integration: an automated process that runs on pushes to main and pull requests. The workflow installs the Go version required by go.mod, starts disposable PostgreSQL and Redis services, applies SQL migrations, runs tests and race checks, runs `go vet`, checks whitespace, builds the Docker image, starts a smoke-test container and probes `/health` and `/ready`.

To learn the configuration, read `.github/workflows/ci.yml` sequentially: trigger events, permissions, services/health checks, environment variables, checkout, Go setup, migration step, test step, vet/whitespace, Docker build, runtime run and curl probes. Change one step at a time and confirm the next CI result.

14.1 DOCKER SMOKE TEST AND SCREENSHOTS

The local smoke test script starts PostgreSQL and Redis with Compose, waits for readiness, builds the Docker image, starts the app container, and invokes `/health` and `/ready`. It loads `.env` locally but deliberately avoids printing its secrets. It uses a host name reachable from the container on macOS and cleans up the temporary API container when finished.

Run the script from the repository root only after confirming the local `.env` has valid DATABASE_URL, JWT_SECRET and MOCK_PAYMENT_WEBHOOK_SECRET values. Keep Postgres/Redis data volumes and the development `orderflow` database intact; the script does not run migrations or drop databases.

Capture a screenshot of the successful Docker build and one showing both `/health` and `/ready` successful responses. Do not capture the environment export or startup logs if they reveal credentials. Save the images under `docs/evidence/docker/` and insert them in the Deployment and Verification chapters with figure captions.

15. AWS DEPLOYMENT FLOW (TARGET ARCHITECTURE)

The AWS figure is a target architecture, not evidence of a live deployment. A possible entry layer is HTTPS through an application load balancer, with the Go API running on EC2 or a managed container service in private application subnets. RDS PostgreSQL should remain private. Redis may use ElastiCache if the assessment justifies the cost and availability requirements. Secrets should be managed by Secrets Manager or Parameter Store; CloudWatch can collect logs and alarms.

Use security groups to allow only necessary traffic: public HTTPS at the entry tier, API-to-RDS on the database port, and API-to-Redis on its port from the application security group. Avoid public database/cache endpoints. Configure database backups, patching, TLS, least-privilege IAM and secret rotation. A single-instance proof-of-concept can be cheaper but has a single point of failure and must be described as such.

Actual AWS deployment requires explicit selection of region, resource sizes, networking, credential setup, budget/cost approval and post-deployment verification. This report does not claim those resources were provisioned.

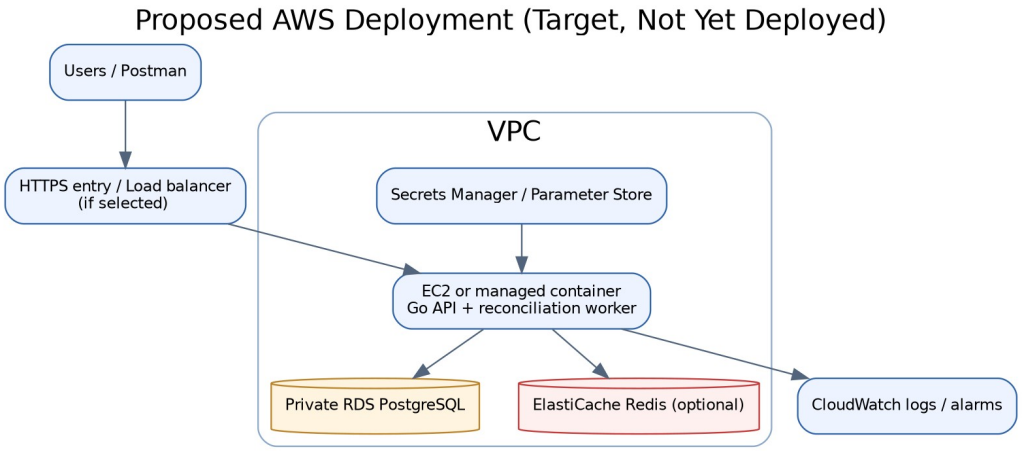


Figure 15.01: Proposed AWS deployment flow (not yet deployed)

15.1 DEPLOYMENT OPERATIONS AND COST CONTROL

Before creating cloud resources, estimate compute, managed PostgreSQL, Redis, data transfer, storage and log retention cost using the selected region and sizes. Use a budget alarm. Stop or delete non-persistent resources after demonstration when appropriate, but preserve database backups and any assessment evidence before cleanup.

Deployment evidence should include the AWS service configuration with sensitive account details redacted, health/readiness responses from the deployed URL, security group rule review, database connectivity from the application, CI commit SHA and a Postman request/response against the deployed API. Screenshots of a proposed architecture alone are not deployment evidence.

If cloud access or budget is unavailable, document the reproducible Docker deployment and the target AWS architecture honestly. Do not claim EC2, RDS, VPC, ElastiCache or CloudWatch resources as live unless they were actually created and tested.

16. TECHNOLOGIES USED

Go is used for typed backend code, context-aware service calls and concurrent execution. Gin provides routing and middleware. PostgreSQL manages relational integrity and transactional state. pgx provides database operations and a connection pool. JWT represents bearer-token identity, and bcrypt protects passwords. Redis is an optional catalog cache. Docker standardizes runtime packaging and Compose runs dependencies locally. GitHub Actions automates validation on code changes.

OpenAPI describes the external API contract and Postman provides an executable manual request collection. Mermaid diagrams are stored as editable sources in the repository and embedded in the README/report. The use of Go rather than the Spring/JPA examples in some course materials is a deliberate project stack choice: the architectural equivalents are DTOs/handlers/services/repositories with explicit SQL/pgx transaction code.

The report should not list tools as implemented when they are merely target components. For example, a proposed AWS RDS/ElastiCache architecture is different from a working AWS deployment, and a mock callback is different from Stripe integration.

Technology	**Role**	**Trade-off / relevance**
Go + Gin	HTTP API and application code	Small deployable service; requires explicit layering and validation.
PostgreSQL + pgx	Durable state and transaction control	Strong integrity; query/transaction design matters.
JWT + bcrypt	Authentication	Token expiry/secret rotation need operational care.
Redis	Optional product read cache	Faster reads; introduces invalidation/availability complexity.
Docker + Compose	Repeatable local and CI runtime	Container correctness depends on network/config setup.
GitHub Actions	Automated build/test gates	CI proves only the commit it runs on.
OpenAPI + Postman	Contract and manual testing	Must be kept in sync with the actual routes.

17. LIMITATIONS, RISKS AND FUTURE WORK

The project is a capstone proof of engineering concepts, not a production commerce platform. The payment provider is mocked; there is no real card flow or payment settlement. The inventory model covers a single warehouse. Shipping, refunds/restocking, multi-currency/tax rules and frontend integration are outside initial scope.

Before final submission, verify high-priority requirement gaps: multiple line items per order; order-level idempotency; amount/currency validation and failure reason; inventory consumption on successful payment or a clearly implemented fulfillment transition; expiry/cancellation stock release; and admin order/sales/inventory reports. The repository's current code has a reconciliation report and inventory movement history but those are not the same as every reporting feature in the initial spec.

Operational limitations include no verified AWS deployment yet, no real provider integration, benchmark numbers pending, and screenshots pending. Redis/worker additions are in a proposed overlay and should not be treated as merged until the test script and GitHub Actions pass. The report must clearly distinguish completed functionality from open requirements.

Gap	Impact	Next action
Multi-product order + order idempotency	Potential mismatch with original must-have requirements	Implement service/DB contract and integration tests.
Payment amount/currency + failure reason	Callback can be weaker than required	Add validation, persistence and negative tests.
Successful payment inventory consumption	May retain reservation instead of consuming stock	Align model and tests with the specification.
Admin sales/order reports	Reporting coverage incomplete	Add protected summary endpoints if rubric requires.
Performance benchmarks	No measured p95 or cache improvement	Run repeatable baseline and post-change workloads.
AWS / real provider	Credential/cost-dependent, not yet deployed	Provision only with explicit approval.

18. CONCLUSION

OrderFlow demonstrates a practical backend design around a shared transactional database: the service owns use-case decisions, repositories own persistence operations, constraints protect core invariants, and tests exercise failure and concurrency paths. Its strongest verified feature is the atomic inventory reservation for order creation and its associated audit movement, supported by PostgreSQL integration and race tests.

The codebase contains the major Go layers already. Additional work should focus on exact acceptance requirements and observable evidence rather than adding empty abstractions. A feature is complete only when implemented, automated tests pass, the commit is pushed, CI is green, documentation matches the behavior, and any required screenshot/benchmark has been captured.

The next steps are to verify the Redis/worker/Docker bundle, close the must-have specification gaps, complete the 40-page formatted report with actual evidence, and only then package the final submission. The AWS target diagram must remain labelled as proposed until a live deployment is validated.

REFERENCES

[1] Go Authors. The Go Programming Language Specification and Documentation. https://go.dev/doc/ (accessed 18 October 2026).

[2] Gin Web Framework. Documentation. https://gin-gonic.com/docs/ (accessed 18 October 2026).

[3] PostgreSQL Global Development Group. PostgreSQL Documentation: Transactions, Constraints and Explicit Locking. https://www.postgresql.org/docs/ (accessed 18 October 2026).

[4] Jack Christensen et al. pgx: PostgreSQL Driver and Toolkit for Go. https://github.com/jackc/pgx (accessed 18 October 2026).

[5] Redis Ltd. Redis Documentation. https://redis.io/docs/latest/ (accessed 18 October 2026).

[6] OpenAPI Initiative. OpenAPI Specification. https://spec.openapis.org/oas/latest.html (accessed 18 October 2026).

[7] Docker, Inc. Dockerfile and multi-stage builds documentation. https://docs.docker.com/build/building/multi-stage/ (accessed 18 October 2026).

[8] GitHub. GitHub Actions workflow syntax. https://docs.github.com/actions/writing-workflows/workflow-syntax-for-github-actions (accessed 18 October 2026).

[9] Jones, M., Bradley, J., Sakimura, N. JSON Web Token (JWT), RFC 7519. IETF, 2015. https://www.rfc-editor.org/rfc/rfc7519.

[10] Fielding, R. et al. HTTP Semantics, RFC 9110. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9110.

[11] OrderFlow project specification and source repository maintained for this capstone. https://github.com/manmohansharma21/orderflow (repository content accessed 18 October 2026).

[12] Scaler Neovarsity / Woolf. Applied Software Project Report Template (Backend Specialization), supplied course template. Use the current institution-provided version for final formatting and review.

APPENDIX A. LOCAL RUN AND VERIFICATION COMMANDS

Run commands from the repository root. Copy .env.example to .env and generate separate JWT_SECRET and MOCK_PAYMENT_WEBHOOK_SECRET values. Never print, upload or commit the secret values. Start PostgreSQL and Redis through Docker Compose, then apply migrations only to a fresh database. For an existing local database, do not rerun the full initial migration loop.

Baseline checks: go test ./...; go test -race ./internal/integration/...; go vet ./...; git diff --check. Residual milestone verification: export ORDERFLOW_TEST_DATABASE_URL to the isolated orderflow_test database; the script starts Redis and sets ORDERFLOW_REDIS_TEST_ADDR. The Docker smoke script requires a valid .env and checks image startup and health/readiness.

Before committing, review git status --short, git diff --cached --check and git diff --cached --stat. Stage go.mod and go.sum if `go mod tidy` legitimately adds the Redis dependency checksums. Do not stage .env, SQL backups, credentials, or runtime tokens.

APPENDIX B. SCREENSHOT CAPTURE AND PASTE CHECKLIST

Capture only actual results from the local machine or the actual CI run; do not use mocked screenshots. Each image should show the relevant command/request and success result, while redacting tokens, passwords, account identifiers and webhook secrets. Prefer PNG format. Store project evidence inside docs/evidence/ with stable descriptive names.

Figure B.01 — GitHub Actions green run: after the Redis/Docker bundle is pushed, open Actions, select the latest run on main, ensure its SHA matches the current HEAD, and capture the successful job summary. Paste into Chapter 12 or Chapter 14.

Figure B.02 — Docker smoke test: run `./scripts/docker-smoke-test.sh`; capture the final Docker build/run output and successful /health and /ready results. Paste into Chapter 14. Figure B.03 — Postman authentication: run register/login/profile using a demo account and variables, redact the token, paste into Chapter 7. Figure B.04 — admin product creation and inventory receipt: paste into Chapter 7/8. Figure B.05 — order create plus inventory movement: paste into Chapter 8/9. Figure B.06 — mock payment success and failure/replay: paste into Chapter 10. Figure B.07 — reconciliation output: paste into Chapter 10. Figure B.08 — benchmark results: only after running a repeatable workload, paste p50/p95, throughput, concurrency, dataset size, duration, cache mode, and environment into Chapter 13.

Figure B.09 — AWS deployment evidence: only if deployed, capture the running service health endpoint and relevant resource configuration with account IDs and secrets redacted; paste into Chapter 15. If not deployed, do not paste a console mockup; retain the target diagram and label cloud deployment as future work.

APPENDIX C. REQUIREMENT-TO-EVIDENCE TRACEABILITY

This table deliberately distinguishes verified code from open requirements. Update the status only after the implementation, tests and artifacts are available on the same final commit.

Requirement group	Current evidence	Remaining check
FR-01–05 Identity	Auth handler, JWT middleware, middleware tests	Capture Postman register/login/profile; verify clean test run.
FR-06–08 Catalog	Product service/repository, routes, tests	Capture admin CRUD and duplicate-SKU conflict.
FR-09–12 Inventory	Inventory service/repository, movements, tests	Capture receipt/adjustment and history.
FR-13–20 Orders	Transactional service, integration/concurrency tests	Check multi-item support and order idempotency against spec.
FR-21–27 Payments	Mock payment service, HMAC callback, replay tests	Add/verify amount-currency match, failure reason and negative tests.
FR-28–30 Completion	Order state changes, failure release, cancellation	Verify successful stock consumption and expiry semantics.
FR-31–32 Reporting	Reconciliation and inventory movement endpoints	Add order/sales/inventory summary if rubric requires.
NFR Performance	Query/index patterns and optional Redis bundle	Record baseline/cache p95 and throughput; do not invent metrics.
Deployment	Dockerfile, workflow and target AWS guide	Verify current CI green; local smoke; live AWS only with approval.
Report/submission	Report draft, HLD/LLD, OpenAPI, Postman collection	Insert screenshots; replace highlighted placeholders; export final PDF.